

例 5.8 的程序运行结果如图 5.11 所示。

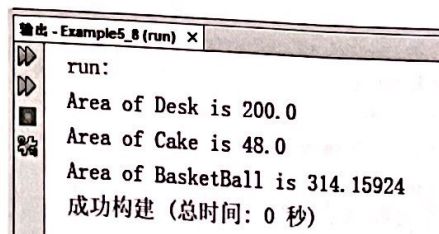


图 5.11 例 5.8 的程序运行结果

5.3.4 接口与多态

5.3.3 节学习了接口回调,即当把实现接口的类的实例的引用赋值给接口变量后,该接口变量就可以回调类重写的接口方法。由接口产生的多态就是指不同的类在实现同一个接口时可能具有不同的实现方式,那么接口变量在回调接口方法时就可能具有多种形态。

在设计程序时,经常会使用接口,其原因是接口只关心操作,但不关心这些操作的具体实现细节,可以使我们把主要精力放在程序的设计上,而不必拘泥于细节的实现。也就是说,可以通过在接口中声明若干个 abstract 方法,表明这些方法的重要性,方法体的内容细节由实现接口的类去完成。使用接口进行程序设计的核心思想是使用接口回调,即接口变量存放实现该接口的类的对象的引用,从而接口变量就可以回调类实现的接口方法。

下面的例 5.9 中,准备设计一个广告牌,希望所设计的广告牌可以展示许多公司的广告词。

首先设计了一个接口 Advertisement,该接口有两个方法: showAdvertisement() 和 getCorpName(),那么实现 Advertisement 接口的类必须重写 showAdvertisement() 和 getCorpName() 方法,即要求各个公司给出具体的广告词和公司的名称。

然后设计 AdvertisementBoard 类(广告牌),该类有一个 show(Advertisement adver) 方法,该方法的参数 adver 是 Advertisement 接口类型(就像人们常说的,广告牌对外留有接口)。显然,该参数 adver 可以存放任何实现 Advertisement 接口的类的对象的引用,并回调类重写的接口方法: showAdvertisement() 显示公司的广告词、回调类重写的接口方法; getCorpName() 显示公司的名称。

详细代码如下。

例 5.9

Example5_9.java

```
public class Example5_9{
    public static void main(String args[ ]){
        AdvertisementBoard board = new AdvertisementBoard( );
        board.show(new PhilipsCorp( ));
        board.show(new LenovoCorp( ));
    }
}
```

```
public interface Advertisement {           //接口
    public void showAdvertisement( );
```

```

        public String getCorpName( );
    }

    //负责创建广告牌
    public class AdvertisementBoard {
        public void show(Advertisement adver) {
            System.out.println("广告牌显示" + adver.getCorpName( ) + "公司的广告词:");
            adver.showAdvertisement( );
        }
    }

    //PhilipsCorp 实现 Advertisement 接口
    public class PhilipsCorp implements Advertisement {
        public void showAdvertisement( ) {
            System.out.println("@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@");
            System.out.println("没有最好, 只有更好");
            System.out.println("@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@");
        }
        public String getCorpName( ) {
            return "飞利浦";
        }
    }

    //LenovoCorp 实现 Advertisement 接口
    public class LenovoCorp implements Advertisement {
        public void showAdvertisement( ) {
            System.out.println("*****");
            System.out.println("让世界变得很小");
            System.out.println("*****");
        }
        public String getCorpName( ) {
            return "联想集团";
        }
    }

```

例 5.9 的程序运行结果如图 5.12 所示。

接口的用处具体体现在下面几方面。

(1) 通过接口实现不相关类的相同行为, 而无须考虑这些类之间的关系。

(2) 通过接口指明多个类需要实现的方法。

(3) 通过接口了解对象的交互界面, 而无须了解对象所对应的类。

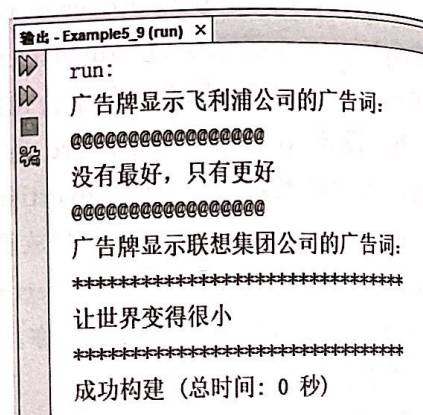


图 5.12 例 5.9 的程序运行结果

5.3.5 抽象类与接口的比较

接口和抽象类的比较如下。

(1) 抽象类和接口都可以有 abstract 方法。

(2) 接口中只可以有常量, 不能有变量; 而抽象类中既可以有常量也可以有变量。

(3) 抽象类中也可以有非 abstract 方法, 接口不可以。

在设计程序时应当根据具体的分析来确定是使用抽象类还是接口。抽象类除了提供需要子类去实现的 abstract 方法外, 也提供了子类可以继承的变量和非 abstract 方法。